Part 9 Brief · Safeguards in the kit

mcp-ops-server · 20 Oct 2026 · PR #20 · Status: built, CI running; waiting for Codex review

WHAT WAS BUILT (IN MCP_OPS_KIT, SO THE CLINIC AND THE SALON BOTH HAVE IT)

- **Tokens file format 2 + audience binding:** every token names its audience, expiry and revocation. The verifier re-reads the file on change, so `revoke` works on a running server. An unversioned file is refused with the `migrate` command. Closes the Part 8 cross-domain P2.
- **OAuth 2.1 resource server:** RS256/ES256 JWT access tokens (RFC 9068) checked against a JWKS (file or URL, cached, rotation-aware), iss/aud/exp/nbf, and a `jti` revocation file. Static and JWT side by side.
- **Audit log (164.312(b)):** one record per tool call (client, token ID, tool, scope, subject ID, outcome, time), never arguments. AES-256-GCM per record, HMAC chain across records, `verify`/`show` CLIs.
- **Encryption at rest (164.312(a)(2)(iv)):** a SQLite file holds only ciphertext. The database lives in memory, and each committed write re-encrypts it atomically (about 3 ms). One data key, HKDF subkeys per purpose.
- **Data minimisation:** every tool's fields pinned with a reason, checked on every backend. The server refuses to start without a token source, an audit log and a key.

HOW IT WORKS

Call path	Safeguard
Bearer token	static: hash, audience, expiry, revoked? / JWT: signature, iss, aud, exp, jti
OpsServer.call_tool	audit begin, then rate limit, before argument validation
Tool: guard.check	scope and subject; notes scope, subject and refusal for the audit record
Store	SQLite in memory, re-encrypted to disk on every committed write
Result	text rebuilt from the validated output; one audit record appended and fsynced

WHY IT WAS DONE THIS WAY

- **Fail closed where a safeguard can fail.** A broken tokens or revocation file refuses every token (keeping the old set could skip a revocation). An unwritable audit record stops the server, but the call already made still returns its real result: reporting it as failed would be false (Part 7's lesson).
- **No native dependency.** SQLCipher needs a C library, and field-level encryption leaks the schema and row counts. One new dependency, `cryptography`, for AES-GCM and the JWT signatures.

WHAT WENT WRONG OR SURPRISED ME (LOGGED IN FAILURES.MD)

- **The agent could read undeclared fields.** The SDK drops them from structured content, but builds the text, which the model reads, from the raw return. I found it when my minimisation test passed against a deliberately leaky backend. The kit now rebuilds the text.
- **Before pushing:** a 0600 tokens file the container could not read (found by replaying CI's container steps locally), a lock leaked on a refused database, an empty JWKS that raised a 500, and old tests that passed for the wrong reason. 14 of 15 deliberate mutations were caught; the survivor is a redundant check.

VERIFY IT YOURSELF

```
pytest -q                                    # 505 passed
python -m evals.run; python -m salon_ops.evals.run   # 31/31 12/12; 18/18 14/14
python -m mcp_ops_kit.audit verify --file audit.log --key-env MCP_OPS_DATA_KEY
```

INTERVIEW TALKING POINT

"My data-minimisation test passed against a backend I had made leak on purpose. The framework stripped the extra field from the structured result but not from the text the model reads. So I test what the consumer sees, not what the framework validates."

NEXT (YOUR ACTIONS)

1) Paste the Codex prompt (in STATUS). 2) Note: the image now needs MCP_OPS_DATA_KEY, and old tokens files need migrate (README). 3) Merge when green. 4) Say **go** for Part 10.